Preprocessing

Classification:

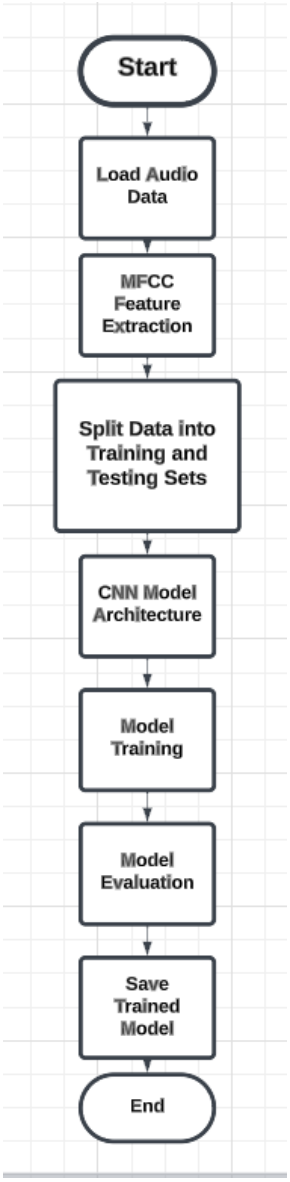
- → **Resampling**: Ensure that all audio samples have the same sampling rate.
- → **Normalisation**: Scale the audio data to have a consistent amplitude within the range of -1 to 1. This is important for consistent input to machine learning models.
- → **Feature Extraction**: Extract relevant features from the audio data such as Mel-Frequency Cepstral Coefficients (MFCCs) or Mel spectrograms
- → **Data Augmentation** (optional): Augment the dataset by applying transformations like time stretching, pitch shifting, or adding noise to increase the variety of training data.
- → **Label Encoding**: Convert class labels (e.g., 'siren' or 'horn') to numerical values, which are required for training machine learning models.

Localisation:

- → **Resampling**: Consistent sampling rates across all microphone signals in the array is needed.
- → **Synchronisation**: Synchronise the captured audio signals to ensure they are aligned in time; this is crucial for accurate Time Difference of Arrival (TDOA) calculations.
- → **Microphone Calibration** (optional): Calibrate microphones to account for variations in sensitivity, phase, and frequency response.
- → **Beamforming** (optional): Apply beamforming techniques to enhance the signal of interest and suppress noise or interference.

Building a CNN plus important subroutines

Objectives 4.1, 4.3



Important libraries for Audio Classification

```
import numpy as np
import librosa
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

Model Training

```
model.fit(X_train, Y_train, epochs = 10,
validation_data=(X_test, Y_test))
```

Model Compilation

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
                        metrics=['accuracy'])
```

Model Evaluation

```
test_loss, test_acc = model.evaluate(X_test, Y_test,
verbose=2)
print("Test accuracy:", test_acc)
```

```
model = Sequential()
model.add(Input(shape=(13, frames, 1))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(3, activation='softmax'))
```

softmax activation is used as it is multi-class classification

3 before represents the total number of classes (sirens, horns, other sounds)

CNN Subroutines:

Name	What does it do?	Returns?
Feature extractor	This function will take an audio file as input, compute its Mel-Frequency Cepstral Coefficients, scale and average them, and will return a feature vector that summarises the audio characteristics in terms of these coefficients. This feature vector can be used for various audio analysis tasks such as speech recognition, music classification, and more.	Mfccs_scaled_features A numpy array containing the scaled and averaged coefficients
Preprocess-audio	It will continuously take live audio as an input, check if the volume is above a specific threshold, and if so, it extracts and preprocesses MFCC features. This will	**Mfcc_scaled_features**

	prove useful as there is always low volume audio that will need to be filtered out.	
Live_audio_classification	This is the main subroutine which performs the classification live. It will first specify the audio format, number of channels, sampling rate and frames per buffer. It will then continuously capture audio chunks until the exit command is detected. It will use the labelEncoder to transform the predicted label back to the class name	**Name of the predicted class e.g. 'siren'**

Psuedocode

```
FUNCTION features_extractor(file_name):

    ### Load audio data from the dataset

    audio, sample_rate = load_audio(file_name)


    ### Extract MFCC features

    mfccs_features = calculate_mfcc(audio, sample_rate)


    ### Calculate the mean of MFCC features

    mfccs_scaled_features = calculate_mean(mfccs_features)


    ### Return the scaled MFCC features

    RETURN mfccs_scaled_features
```

Preprocess_Audio:

```
FUNCTION preprocess_audio(audio_data, sample_rate, volume_threshold):
    ### Convert audio data to floating-point format [-1, 1]
    audio_data = normalize_audio(audio_data)

    ### Calculate the volume
    audio_volume = calculate_audio_volume(audio_data)

    ###  Check if the volume exceeds the threshold
    IF audio_volume >= volume_threshold:
        mfccs_features = calculate_mfcc(audio_data, sample_rate)
        mfccs_scaled_features = calculate_mean(mfccs_features)
        RETURN mfccs_scaled_features
    ELSE
        RETURN None        ### Audio volume is below the threshold

FUNCTION normalise_audio(audio_data):
    ### Normalise audio data to the range [-1, 1]
    RETURN audio_data / max(abs(audio_data))

FUNCTION calculate_mfcc(audio_data, sample_rate):
    ### Calculate MFCC features from audio data and sample rate
    mfccs_features = mfcc(audio_data, sample_rate, n_mfcc=40, n_fft=1024)
    RETURN mfccs_features
```

Live audio classification

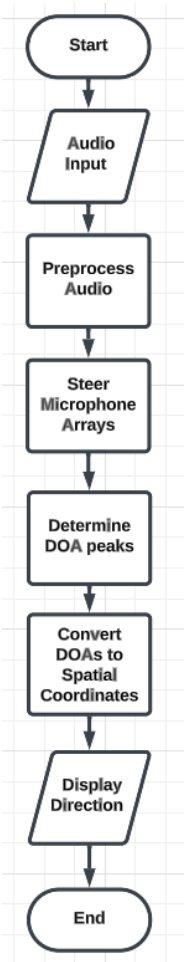
```
FUNCTION live_audio_classification(volume_threshold):
  pyaudio_instance = create_pyaudio_instance()
  audio_stream = open_audio_stream(pyaudio_instance)
  ### Print a message indicating the program is currently active
  OUTPUT  "Listening for audio... (Press 'e' to exit)"
### Continuous loop to listen for and classify audio
  WHILE True:
    TRY:
      ### Read a chunk of audio data from the audio stream
      audio_chunk = read_audio_chunk(audio_stream)
      ### Convert audio data to a NumPy array
      audio_data = convert_audio_chunk_to_array(audio_chunk)
      ### Check if 'e' key is pressed to exit
      IF 'e' key is pressed:
        OUTPUT "Exiting..."
        Break out of the loop
      ### Preprocess audio data if it exceeds the volume threshold
      mfccs_scaled_features = preprocess_audio(audio_data, sample_rate=44100, volume_threshold)
       IF mfccs_scaled_features ≠ None:
        ### Predict the class label using a pre-trained model
        predicted_label = predict_class_label(mfccs_scaled_features)

        ### Convert the predicted label to a class name
        predicted_class = convert_to_class(predicted_label)
        OUTPUT "Predicted Class: {predicted_class}"

  close_audio_stream(audio_stream)
  terminate_pyaudio_instance(pyaudio_instance)
```

Algorithms

The DOA (direction of arrival) algorithm will come in-built with the microphone array that I will purchase, however I thought it was important to analyse how they work in the background and what different techniques may be implemented in their DOA algorithm. I will explore a couple of the most common DOA yet robust DOA algorithms.

Objective 3.1

DOA estimation with SRP-PHAT Algorithm



Preprocess Audio: contains 2 important processes, applying the Short Time Fourier Transform and computing the Cross Correlation Matrix

STFT – a useful technique to understand how the frequency of a moving sound source changes over time. Short segments of the signal are taken, and the Fourier Transform are applied to each of them. A time frequency spectrogram is usually produced, where the colour of each point represents the intensity of the signal at the time.

Cross Correlation Matrix - the similarity of signals between 2 microphones is measured after one wave is time shifted over the other. This similarity is measured for each microphone pair and is stored in a matrix. A higher value represents higher similarity between signals.

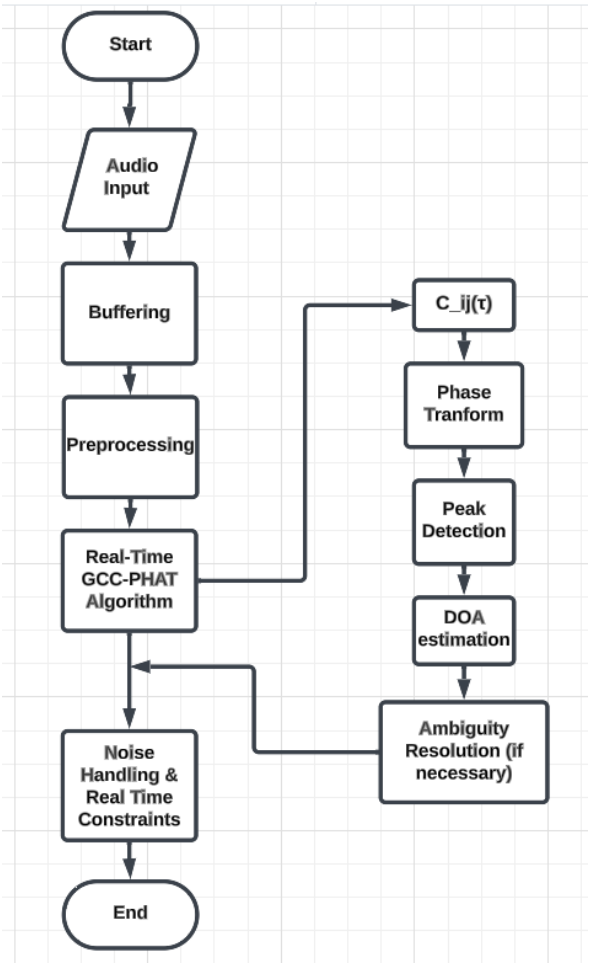
Steer Microphone Arrays: contains 2 important subroutines that will be called separately, Apply Phase Transform (PHAT) and Compute Steered Response Power

Phase Transform - this function aligns the information across all microphone pairs to ensure all pairs correspond to the same DOA. A weighted summation is then taken to emphasise the phase information while suppressing noise/interference

Steered Response Power - microphone are pointed to every direction as if they were directional. A response power is calculated for each angle based on how well the microphone array responds to that certain direction. A power spectrum is formed indicating the strength of incoing signals at different angles

DOA estimation with GCC-PHAT algorithm

Objective 3.1



Buffering – when working with real-time audio data, sound is captured in a continuous stream. To process the audio, the sounds are sampled and temporarily stored in the buffer, a queue data structure, where new frames are appended, and the first frames are deleted after having been processed. A larger buffer size can provide more data for analysis but can introduce more latency.

Calibration – a step that could come after buffering for safety critical systems where each microphone is calibrated so they each have the same sensitivity and gain. Array geometry and system calibrations can also take place to ensure accurate knowledge of microphone positions and orientations as well as their interactions with ADCs (analog to digital converters) and amplifiers.

Preprocessing: this subroutine could call 2 further sub subroutines, Noise Reduction and Bandpass filtering
Noise Reduction – noise can significantly affect the accuracy of localisation algorithms, which is where techniques such as simple noise floor estimation or more advanced methods such as spectral subtraction and adaptive filtering can be used to estimate and characterise the noise. Then techniques such as Weiner Filtering can be performed for noise reduction, based on the estimated signal to noise ratio.
Bandpass Filtering – after preforming the Fourier Transform, applying a bandpass filter to each frame allows only the frequency range of interest to pass through, therefore reducing redundant data from being further processed.

GCC-PHAT Algorithm:

C_ij(τ) - represents the cross-correlation function between 2 adjacent microphones given they have a certain time delay **τ**. Cross correlation measures the similarity of 2 signals after one has been shifted over the other.

PSUEODOCODE

```
IMPORT numpy as np

### 2 audio signals received by adjacent microphones

mic1_signal = np.array[0.1,0.2,0.3,0.4]

mic2_signal = np.array[0.3,0.5,0.3,0.2]

### calculate length of the signals (only take first one to ensure both are same length so that they are
aligned for cross-correlation

signal_length = LEN(mic1_signal)

### initialising array to store cross-correlation values

cross_correlation = np.zeroes(2*signal_length - 1)

### compute cross corrrelations

FOR lag IN RANGE (-signal_length+1, signal length)

    cross_correlation[lag + signal_length - 1] = np.sum(signal_mic1 * np.roll(signal_mic2, lag))
```

Phase Transform Algorithm

```
### Calculate the magnitude of the cross-correlation value

  magnitude = ABS(cross_correlation)


  ### Calculate the Phase Transform by dividing the cross-correlation value by its magnitude

  IF magnitude != 0:

    phase_transform[lag + signal_length - 1] = cross_correlation / magnitude

  ELSE:

    ### Handle division by zero (to avoid NaN)

    phase_transform[lag + signal_length - 1] = 0

### Output:

### phase_transform: Array containing the Phase Transform values
```

Peak detection Algorithm

```
### Inputs:
### phase_transform: Phase Transform array
max_phase_value = -INFINTY
max_phase_lag = 0
### Finding the peak in the Phase Transform
FOR lag in range(0, 2 * signal_length - 1):
   IF phase_transform[lag] > max_phase_value:
      max_phase_value = phase_transform[lag]
      max_phase_lag = lag
### Output:
### max_phase_lag: The lag corresponding to the peak in Phase Transform
```

DOA calculations

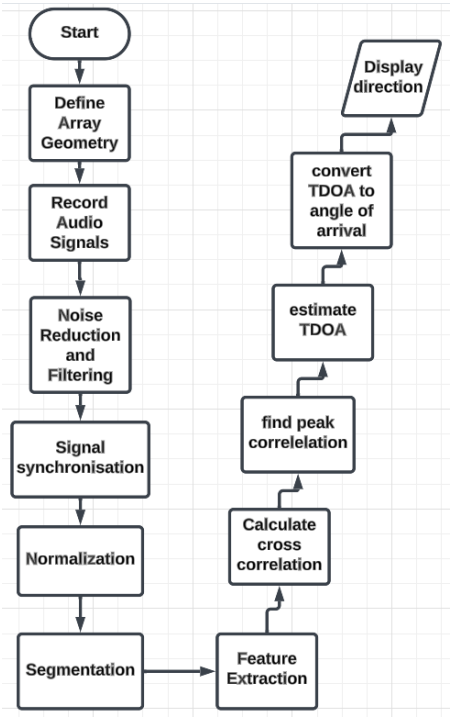
```
DOA for Linear Microphone Array
### Inputs:
# max_phase_lag: Lag corresponding to the peak in Phase Transform
# speed_of_sound: Speed of sound in the medium
# microphone_spacing: Distance between adjacent microphones
# array_length: Number of microphones in the array
# Calculate the time delay from the peak detection
time_delay = max_phase_lag / 2.0
### Calculate the direction of arrival (DOA) in radians
wavelength = speed_of_sound / (2 * microphone_spacing)
angle_rad = ARCSIN(time_delay * wavelength)  ### Calculate the angle using arcsin
### Convert the angle to degrees if needed
angle_deg = degrees(angle_rad)
OUTPUT:
angle_deg   ###Direction of Arrival in degrees
```

DOA for 4 mic circular array

```
### Inputs:
# max_phase_lag - Lag corresponding to the peak in Phase Transform
# speed_of_sound - Speed of sound in the medium
# microphone_spacing - Distance between adjacent microphones
# radius: Radius of the circular array
# Calculate the time delay from the peak detection
time_delay = max_phase_lag / 2.0
### Calculate the angle (in radians) based on the time delay
wavelength = speed_of_sound / (2 * microphone_spacing)
angle_rad = ARCSIN(time_delay * wavelength)
### Calculate the DOA in radians using the circular array geometry
### Since this is a square formation, the microphones are at 0, 90, 180, and 270 degrees from the center.
### Calculate the relative angle of the peak from the center of the square array.
relative_angle = angle_rad MOD (2 * π)
### Convert relative angle to an absolute angle within the square array's coordinate system
IF relative_angle < π / 4:
   angle_deg = degrees(relative_angle)
ELSE IF relative_angle < 3 * π / 4:
   angle_deg = degrees(π / 2 - relative_angle)
ELSE IF relative_angle < 5 * π / 4:
   angle_deg = degrees(relative_angle - π)
ELSE:
   angle_deg = degrees(2 * π - relative_angle)
OUTPUT
angle_deg      ### Direction of Arrival in degrees within the square array
```

Localisation with TDOA (time difference of arrival)

Objective 3.1



Noise reduction: techniques such as spectral subtraction and adaptive noise cancellation can be used
Filtering: use bandpass or high pass filters to focus only on the desired frequencies
Synchronisation: ensures that signals have the same start point and are sampled consistently
Amplitude Normalization: ensures amplitudes are all on the same scale – important for further processing
Spectral Analysis: calculate the short Fourier transform of each frame to enhance TDOA accuracy
Features: spectral peaks or energy distribution can be extracted

Psuedocode

```
### Define constants

speed_of_sound = 343

microphone_spacing = "WHATEVER THE MIC SPACING IS"


FUNCTION  estimate_tdoa(reference_microphone, target_microphone, signal):

   cross_correlation = calculate_cross_correlation(reference_microphone_signal, target_microphone_signal)

   tdoa = find_peak_tdoa(cross_correlation)

   RETURN tdoa


### Converting TDOA to angle of arrival

FUNCTION tdoa_to_angle(tdoa):

   angle_radians = ARCTAN(tdoa * speed_of_sound / microphone_spacing)

   angle_degrees = radians_to_degrees(angle_radians)

  RETURN angle_degrees
```

```
### Main program
    reference_microphone_signal = record_audio(reference_microphone)
    target_microphone_signal = record_audio(target_microphone)

    tdoa = estimate_tdoa(reference_microphone_signal, target_microphone_signal)
     angle_of_arrival = tdoa_to_angle(tdoa)

    OUTPUT ("Estimated Angle of Arrival: ", angle_of_arrival, " degrees")
```

Links objectives 2.9, 3.5, 4.7 – Threading and Queues

This pseudocode demonstrates the initialisation of 2 separate queues in which data will constantly be added. Functions are created that are then put into separate threads in order to run simultaneously.

```
### separate queues will be created for DOA values and Prediction classes
DOA_queue = create_queue()
Prediction_class_queue = create_queue()

### Function to put DOA values into the DOA queue
FUNCTION process_doa_data(doa_data):
   DOA_queue.enqueue(doa_data)

### Function to put Prediction classes into the Prediction class queue
FUNCTION process_prediction_data(prediction_data):
   Prediction_class_queue.enqueue(prediction_data)

### Example usage
doa_data = get_doa_data()
prediction_data = get_prediction_data()

### Process DOA values and Prediction data concurrently
start_thread(process_doa_data, doa_data)
start_thread(process_prediction_data, prediction_data)
```

Links to objectives 3.6 and 4.8 – Thread Locking on queues

This pseudocode demostrates the intialisation of locks for both queues and provides an example of how enqueuing works with locking and unlocking the queues in each thread. Locks are mainly used to prevent race conditions and to aid synchronisation between queues. The former is when 2 different threads both try to access the same data simultaneously which could lead to data corruption. Locks also help different threads coordinate their actions (synchronisation).

```
### Create locks for the DOA and Prediction class queues
DOA_queue_lock = create_lock()
Prediction_class_queue_lock = create_lock()

### Acquire lock before enqueueing to the DOA queue
  DOA_queue_lock.acquire()
  try:
    DOA_queue.enqueue(doa)
  finally:
    ### Release the lock after enqueueing
    DOA_queue_lock.release()

### Acquire lock before enqueueing to Prediction class queue
  Prediction_class_queue_lock.acquire()
  try:
    Prediction_class_queue.enqueue(prediction_class)
  finally:
    ### Release the lock after enqueueing
    Prediction_class_queue_lock.release()
```

Link to objective 4.9 – Volume threshold and RMS energy

As I'm working with audio data, I can calculate the root mean square energy of the audio which can easily be compared against a threshold value to create a volume threshold for the live audio classification.

```
### Function to calculate the RMS energy of an audio signal
FUNCTION calculate_RMS_energy(audio_signal):
  sum_of_squares = 0

  ### Calculate the sum of squares of each sample
  FOR sample in audio_signal
    sum_of_squares += sample^2

  ### Calculate the mean of the squared values
  mean_squared = sum_of_squares / length(audio_signal)

  ### Calculate the RMS energy by taking the square root of the mean squared value
  rms_energy = sqrt(mean_squared)
  return rms_energy

### check if audio signal exceeds a specified volume threshold
FUNCTION is_above_threshold(audio_signal, threshold):
  rms_energy = calculate_RMS_energy(audio_signal)
  IF rms_energy > threshold:
```

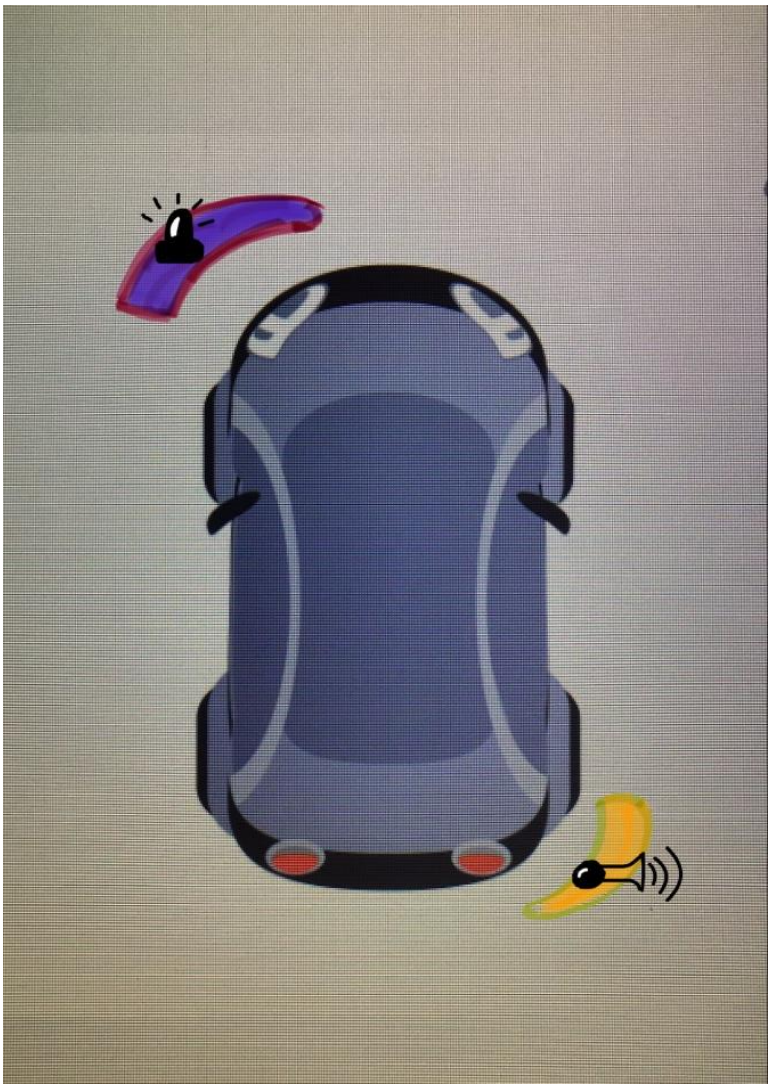
```
    return true
  ELSE
    return false

### Example usage
audio_signal = get_audio_signal()
threshold_value = 0.3    ### Adjust this threshold value according to requirements

IF is_above_threshold(audio_signal, threshold_value):
  start_audio_classification()
ELSE
  OUTPUT("Audio volume is below the threshold.")
```

Link to objective 4.10 – Data Augmentation techniques

```
import numpy as np
import librosa

### Function to add noise to audio
FUNCTION add_noise(audio, noise_level):
  noise = np.random.normal(0, noise_level, len(audio))
  noise_added_audio = audio + noise
  RETURN noise_added_audio

### Function to perform pitch shift
FUNCTION pitch_shift(audio, sr, n_steps):
  pitch_shifted_audio = librosa.effects.pitch_shift(audio, sr, n_steps)
  RETURN pitch_shifted_audio

### Function to perform time stretch
FUNCTION time_stretch(audio, stretch_rate):
  time_stretched_audio = librosa.effects.time_stretch(audio, stretch_rate)
  RETURN time_stretched_audio


### How to use these functions

audio_data, sample_rate = librosa.load("path_to_audio_file.wav", sr=None)

### Add noise
noise_level = 0.002  # Adjust the noise level as needed
noise_added_audio = add_noise(audio, noise_level)

### Pitch shift
semitones = 3  ### Adjust the pitch shift steps as needed
pitch_shifted_audio = pitch_shift(augmented_audio, sample_rate, semitones)

### Time stretch
stretch_rate = 1.3  ### Adjust the time stretch rate as needed
time_streteched_audio = time_stretch(augmented_audio, stretch_rate)
```

User interface

Stakeholder Input:
After sending the clients a photo of the initial design of the user interface, they had a good idea of what they should expect to see whilst driving. I then asked for feedback on the design to see if they came up with any suggestions or complaints.



I sent them the following email:

"Hi,
Please find attached an image of how the user interface could look. I've made sure that the arcs are large and clearly indicate the direction of the sound, as well as ensuring that the icon corresponds to the type of sound being picked up by microphones. Currently the UI doesn't require any input from users, however this can be changed if necessary. Please let me know if you would like anything to be added, changed or improved.
Thanks.
Arzaan"

Here is the response I got from my client:

"Hi Arzaan,

Thanks for the initial UI design. Overall, I think it's fantastic, it clearly has everything that I had asked for. The direction from which sound is coming from is very clear, with vibrant colour arcs that help distinguish that different sounds are being shown. Good to see a nice white background. I like the addition of the symbol alongside the arc to make it even more clear what the type of sound is, and it's good that you chose an appropriate size for it so that I can easily see it and make out the direction clearly. As I've said before, if doing arrows is easier for you then I don't mind if you replace the arcs, as long as it's as neat and professional looking as this UI. For potential improvements, you could perhaps make the indicators dynamic; maybe have the siren one flash with the colours of a siren or something like that.
Thanks for that.
Kind regards
Delzad"

Objectives 2.2, 2.3, 2.4, 2.5, 2.6, 2.7 met above as seen in client's response

UI Output:

Data item	Data type	Optional or required
Central vehicle on display	image	- Optional -
Arcs/arrows	image	- Required - Must be different colours for horn or siren - Must contain icon/text with arc/arrow
Type of sound and direction	Text/image	- Optional - Must be concise - Can print siren or horn - Can use an icon for siren or horn - Could print direction: front, back, left, right, front left, front right, back left, back right

Hardware Selection/Design:

Objective 3.2

Selecting appropriate hardware for finding the direction of arrival of sounds is crucial to the success of the project. The choice of hardware impacts the system's performance, robustness, and real-world applicability. Firstly, the microphone selection is very important; they must be high-quality, omnidirectional (can pick up sounds in 360 degrees) microphones with low self-noise to ensure accuracy of sound capture. Secondly, the physical layout of the microphones is vital. Appropriate geometries must be selected (linear, planar, 3D) according to the scope of the task that needs to be completed. In my case, I am mainly interested in purely 2-dimensional audio since the height of the sound in respect to the car will not be displayed. Therefore, a planar or linear arrangement should be more than enough. However, as I want 360-degree coverage, a planar array would be more suitable. Knowing the distance between microphones as well as the angle between them for non-linear arrays is also important as these data points form the basis of the algorithms that convert the estimated time delay to the approximate direction of arrival.
As of right now, there are a couple of viable microphone array options such as the QuadMic 4-microphone array, the ReSpeaker mic array v2.0 and the ReSpeaker Quad microphone.
A Raspberry Pi could potentially be used if I decide to test the system outside of using my main PC however this will only be done if there is extra time as the audio classification model will need to be converted to TensorFlow Lite for it to run on an embedded system.

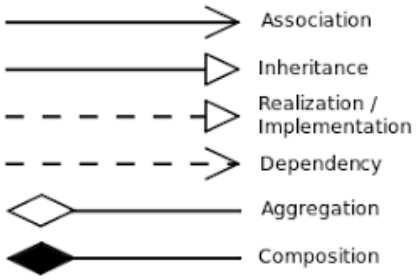
Key Variables

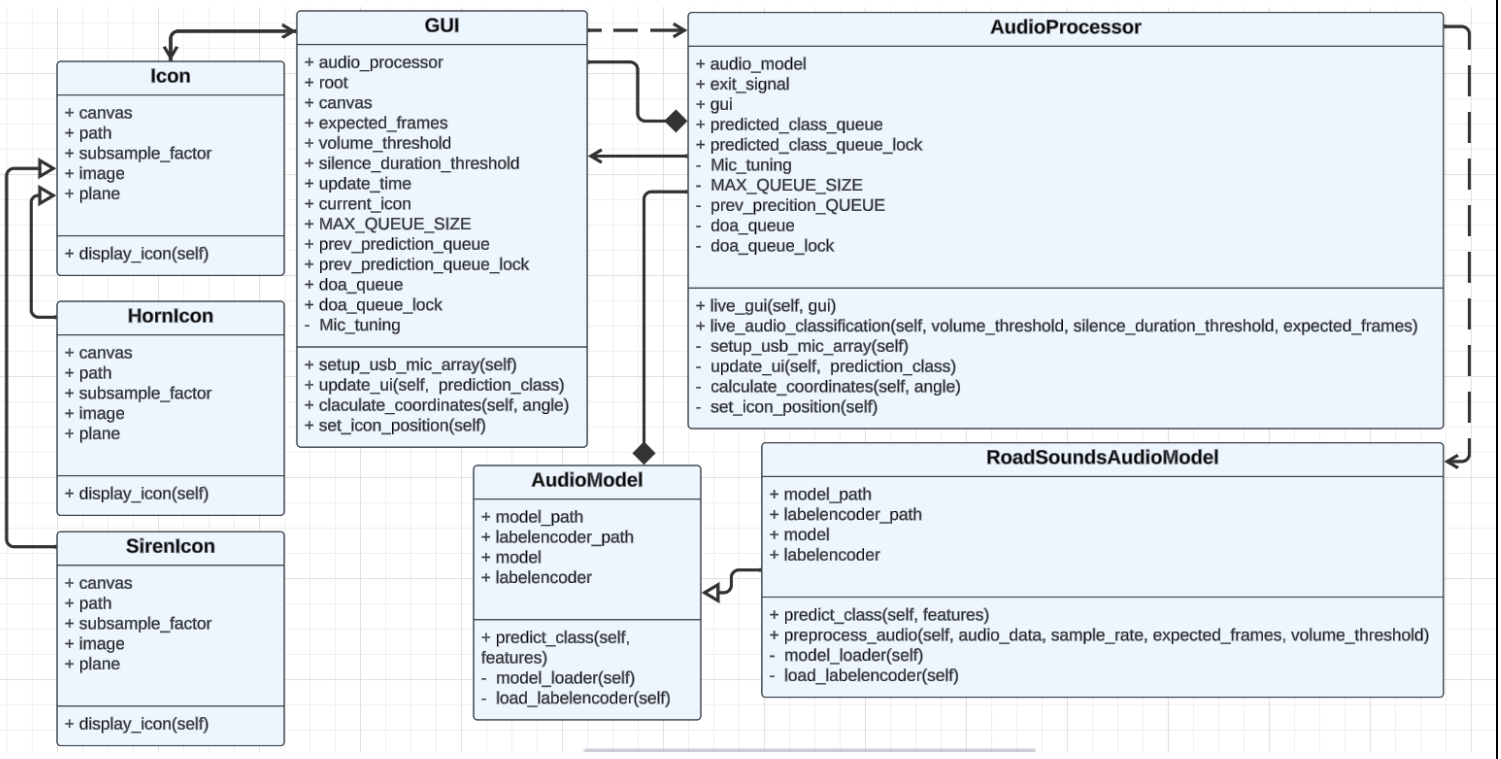
Variable name	**Data type**	**Description**
Classification		
Classification_result	string	The result of audio classification ('siren', 'horn' etc)
Confidence_score	float	Numerical value representing the certainty level of the classification
Audio_signal	1D array	Raw audio from microphone
Preprocessed_signal	1D array	Audio signal after preprocessing (which can include resampling, normalisation and feature extraction)
Model_parameters	dictionary	A set of hyperparameters, model architecture, and weights used in the CNN
Training_data	List of audio samples and labels	Dataset used for training CNN
Feature_vectors	list	Extracted feature vectors (MFCCs)
Extracted_features	Data frame	A data frame that will be made to easily store each audio files' features and the class it belongs to (type of sound). This data structure also allows heterozygous data types to be stored together. This will be used to easily manipulate data if needed.
Class_labels	Numpy array	Extracts class labels from the class column of the data frame. It contains the target classes the model aims to predict.
Audio_features	Numpy array	This will store extracted audio features from the data frame. These will be the input features to the model that will be used to make predictions.
Localisation		
TDOA_matrix	2D array	Matrix containing tdoa between pairs of microphones
DOA	float	Estimated angle from which sound is coming from in degrees or radians

Microphone_positions	List of tuples	Physical positions of microphones in a circular array
DOA_uncertainty	float	Measure of uncertainty associated with doa estimate
Real_time_audio_stream	buffer	Continuous stream of audio for localisation
Display/UI		
DOA_display	float	Doa to be displayed on plot
Sound_type_display	string	Type of sound being heard, to be displayed on the UI near the doa
Plot_data	list/array	Data representing 360-degree plot
Ui_parameters	dictionary	Configuration settings (including plot colours, labels etc)
Update_frequency	Integer	The rate at which the UI will update with new data to plot (in milliseconds)

Class Diagram for Main Application

This class diagram highlights the types of relationships that will exist between different classes of my system. This includes all the classes including public and private attributes and Methods (Public = +, Private = -). Attributes at the top, methods at the bottom.





RoadSoundsAudioModel has an inheritance relationship with AudioModel as it is a specialised version of it. Using the syntax – Class RoadSoundsAudioModel(AudioModel):, it will inherit all its attributes and methods. This is a form of polymorphism.

Both SirenIcon and HornIcon inherit from the parent class Icon as they are specialised versions of the Icon class. This is a form of polymorphism.

The AudioProcessor class is instantiated within the GUI class. This is a composition relationship since a GUI object contains an instance of AudioProcessor. The lifetime of the audio_processor instance is managed by GUI.

GUI also has a dependency on AudioProcessor, the instance is necessary for GUI to interact with audio processing functions.

AudioProcessor has a composition relationship with AudioModel as it contains an instance of AudioModel. AudioProcessor uses functionalities provided by AudioModel and it manages its life cycle.

AudioProcessor has a dependency relationship with RoadSoundAudioModel as it requires functions from RoadSoundAudioModel in order to function.

Attribute	**Data Type**
root	N/A
canvas	Tk.canvas
Expected_frames	int
Volume_threshold	float
Silence_duration_threshold	float
Update_time	int
Current_icon	N/A
MAX_QUEUE_SIZE	int
Prev_prediction_queue	lifoqueue
Doa_queue	queue
Doa_queue_lock	Threading. Lock
Exit_signal	Threading.Event
Predicted_class_queue	queue
Predicted_class_queue_lock	Threading. Lock
path	string
Subsample_factor	int
image	Tk.image

Method	**Parameter Data Type**	**Return variable and its Data Type**
Predict_class(self, features)	Np.float32	Numpy array
Model_loader(self)	N/A	N/A
Load_labelencoder(self)	N/A	N/A
Preprocess_audio(self, audio_data, sample_rate, expected_frames, volume_threshold)	Np.int16, int, int, float	Mfccs_scaled_features – np.float32
Live_gui(self, gui)	N/A	None
Live_audio_classification(self,volume_threshold, silence_duration_threshold, expected_frames)	Float, float, int	None
Setup_usb_mic_array(self)	N/A	None
Update_ui(self, prediction_class)	String	None
Calculate_coordinates(self, angle)	int	Tuple of 2 floats
Set_icon_position(self)	N/A	None
Display_icon(self)	N/A	None